

22 July 2025

A Comparative Study of HNSW Implementations for Scalable Approximate Nearest Neighbor Search

Harshit Shah¹

1. BITS Pilani

Abstract

Hierarchical Navigable Small World (HNSW) graphs are among the most widely adopted algorithms for an approximate nearest neighbor (ANN) search in high-dimensional data, supporting applications across machine learning, recommendation, and computer vision. Multiple open source libraries including FAISS-HNSW, HNSWlib, and NMSLIB-HNSW implement HNSW with different design choices and performance characteristics, yet comprehensive comparisons between them remain limited. This paper presents an empirical benchmarking study of these three libraries, systematically varying the size of the data set, the vector dimension, and key HNSW parameters (ef_construction and M). We evaluated each library on four critical metrics: indexing time, memory usage, query latency, and recall. Our results reveal distinct trade-offs: NMSLIB-HNSW provides the fastest index construction and highest recall, but suffers from significantly slower query times at scale. FAISS-HNSW offers the most balanced overall performance, while HNSWlib excels in smaller-scale settings. All benchmarks were conducted on CPU-based synthetic in-memory datasets. The findings offer actionable guidance for practitioners selecting and tuning HNSW libraries for varied deployment scenarios.

Keywords

approximate nearest neighbor search, benchmarking, computing and processing, faiss, general topics for engineers, high-dimensional data, hns, indexing algorithms, machine learning, nmslib, vector databases

A COMPARATIVE STUDY OF HNSW IMPLEMENTATIONS FOR SCALABLE APPROXIMATE NEAREST NEIGHBOR SEARCH

Harshit Shah

BITS Pilani, Goa Campus
f20170557g@alumni.bits-pilani.ac.in
shahharshit20@gmail.com
ORCID: 0009-0007-6570-2950

ABSTRACT

Hierarchical Navigable Small World (HNSW) graphs are among the most widely adopted algorithms for an approximate nearest neighbor (ANN) search in high-dimensional data, supporting applications across machine learning, recommendation, and computer vision. Multiple open source libraries including FAISS-HNSW, HNSWlib, and NMSLIB-HNSW implement HNSW with different design choices and performance characteristics, yet comprehensive comparisons between them remain limited. This paper presents an empirical benchmarking study of these three libraries, systematically varying the size of the data set, the vector dimension, and key HNSW parameters (ef_construction and M). We evaluated each library on four critical metrics: indexing time, memory usage, query latency, and recall. Our results reveal distinct trade-offs: NMSLIB-HNSW provides the fastest index construction and highest recall, but suffers from significantly slower query times at scale. FAISS-HNSW offers the most balanced overall performance, while HNSWlib excels in smaller-scale settings. All benchmarks were conducted on CPU-based synthetic in-memory datasets. The findings offer actionable guidance for practitioners selecting and tuning HNSW libraries for varied deployment scenarios.

Keywords Approximate Nearest Neighbor (ANN) · Hierarchical Navigable Small World (HNSW) · ANN Benchmarking · Vector Search · Similarity Search · Query Latency · Recall

1 Introduction

The approximate nearest neighbor (ANN) search is a foundational operation in large-scale information retrieval, recommendation systems, computer vision, and natural language processing. As data sets grow in size and dimensionality, traditional exact search methods become computationally prohibitive, driving the adoption of efficient ANN algorithms. Among these, the Hierarchical Navigable Small World (HNSW) graph algorithm has become one of the most popular approaches due to its ability to balance high recall with fast query times and scalability.

To support diverse use cases and platforms, several open source libraries implement HNSW, including FAISS-HNSW, HNSWlib, and NMSLIB-HNSW. Each library offers different engineering choices, default parameters, and ecosystem integrations, which can significantly affect real-world performance. Despite the widespread use of these libraries, there is limited literature that systematically compares their behaviors across key metrics and under varying workload conditions.

This study addresses this gap by providing a comprehensive benchmark analysis of FAISS-HNSW, HNSWlib, and NMSLIB-HNSW. We systematically vary dataset sizes, vector dimensions, and core HNSW parameters (ef_construction and M), and measure four critical metrics: indexing time, memory usage, query latency, and recall. Our goal is to provide actionable insights and practical recommendations for practitioners and researchers tasked with selecting and tuning HNSW based libraries for their specific application needs.

2 Data Availability

All code and data used in this study are publicly available at https://github.com/harshit7999/hnsw_comparison. [12]

3 Related Work

Hierarchical Navigable Small World (HNSW) graphs, introduced by Malkov and Yashunin [1], have become a foundational technique for approximate nearest neighbor (ANN) search, especially in high-dimensional vector spaces. Their logarithmic search complexity and high recall have made them popular in large-scale applications.

Multiple libraries implement HNSW with different design goals. HNSWlib [1] focuses on simplicity and efficiency, FAISS-HNSW [2] integrates with the broader FAISS indexing ecosystem, and NMSLIB [3] emphasizes flexibility across distance functions and indexing strategies.

Several benchmarking efforts have explored the performance of ANN across algorithms. ANN-Benchmarks [4], for instance, provides a standardized evaluation framework to compare various ANN algorithms, including HNSW variants. However, these evaluations often focus on default parameters, a narrow set of dataset scales, or specific metrics such as recall or query time. Other studies have focused on single library optimizations or evaluations on GPU and specialized hardware. Yet, direct empirical comparisons of HNSWlib, FAISS-HNSW, and NMSLIB-HNSW under a controlled and reproducible framework remain limited, particularly with respect to detailed sweeps over HNSW parameters such as `ef_construction` and `M`.

This work fills that gap by systematically comparing these libraries across dataset sizes, vector dimensions, and parameter configurations, while evaluating multiple performance dimensions including indexing time, memory usage, query latency, and recall.

4 Methodology

4.1 Libraries Evaluated

This study focuses on three widely used HNSW implementations.

- **FAISS-HNSW:** Provided by the FAISS (Facebook AI Similarity Search) library, known for its scalability and integration with various indexing methods. [2]
- **HNSWlib:** A lightweight C++/Python library authored by the original HNSW creators, designed for simplicity and ease of integration. [1]
- **NMSLIB-HNSW:** Part of the Non Metric Space Library (NMSLIB), this implementation emphasizes flexibility and performance across a broad range of similarity search tasks. [3]

4.2 Experimental Setup

4.2.1 Hardware and Software

All experiments were performed on a dedicated server equipped with an Intel Xeon Processor (Cascade Lake) with 72 virtual CPUs, 36 GB RAM, and running Ubuntu 24.04 LTS (Linux kernel 6.8.0). Each HNSW library was evaluated using the latest stable versions available at the time of experimentation (June 2025): `faiss-cpu==1.11.0`, `hnsplib==0.8.0`, and `nmslib==2.1.2`. All experiments were executed in a Python 3.12.3 environment, with key dependencies including NumPy, memory profiler, and Matplotlib for data generation, benchmarking, and analysis.

4.2.2 Dataset Generation

Synthetic datasets were generated with the following properties:

- **Number of vectors:** 10,000; 50,000; 100,000; 500,000
- **Vector dimensions:** 100; 200; 300; 500

Each vector was sampled from a uniform random distribution to ensure consistency and avoid dataset bias.

4.2.3 HNSW Parameter Tuning

The following HNSW parameters were systematically varied:

- **`ef_construction`:** 50, 100, 150, 300 (controls the breadth of the graph construction and impacts both index quality and build time)
- **`M`:** 8, 12, 16, 32 (defines the maximum number of neighbors per node in the HNSW graph)

For each parameter combination, `M` and `ef_construction` were explicitly set, while other settings were left to library defaults.

4.2.4 Evaluation Metrics

We measured four key performance metrics:

- **Indexing Time:** Time required to construct the HNSW index
- **Memory Used:** Peak increase in RAM usage measured before and after index construction
- **Querying Time:** Total time to process queries for 10% of the dataset (i.e. 10% of the vectors were used as queries against the index)
- **Recall@k:** Proportion of retrieved neighbors that match the true nearest neighbors, averaged across all queries. We used k=10 in all experiments.

4.3 Implementation Details

All benchmarks were implemented using official APIs in Python 3.12.3 with reproducibility in mind. For each experiment, synthetic data sets and query sets were generated with consistent random seeds to ensure comparability across all combinations of parameters.

For each unique combination of dataset size, vector dimension, ef_construction, and M, we recorded the four evaluation metrics. Recall@10 was computed using an exact IndexFlatL2 search from FAISS as the ground truth.

Each configuration was run three times, and the maximum value (excluding outliers) was used for each metric to account for performance variance. All results were stored in CSV format and summary tables for analysis and visualization were manually generated from these files.

5 Data Analysis

5.1 Indexing Time

5.1.1 Vector Dimension vs. Indexing Time

Table 1: Vector Dimension vs Indexing Time (sec) (varying number of vectors) [ef_construction=150, M=16]

Num of Vectors	Dimension	Indexing Time (sec)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
10,000	100	0.1436	0.8415	0.0971
	200	0.1962	1.3371	0.1628
	300	0.2853	1.9584	0.2062
	500	0.3687	3.2329	0.3019
50,000	100	0.9951	6.1740	0.6935
	200	1.4490	10.6751	1.0252
	300	2.3776	16.3359	1.4314
	500	4.8166	27.2456	2.3567
100,000	100	2.2739	14.5105	1.4570
	200	4.4077	26.4184	2.5743
	300	7.7596	40.5838	3.9808
	500	14.0627	64.6539	6.4599
500,000	100	19.4652	104.8404	14.4362
	200	51.7080	185.7538	24.7186
	300	77.3202	272.6696	33.6683
	500	124.7015	420.5514	51.1286

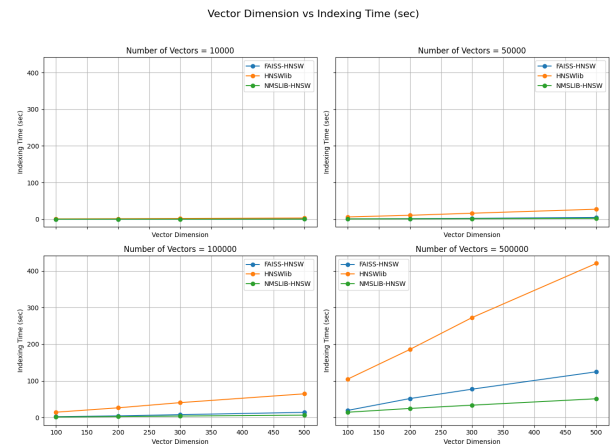


Figure 1: Vector Dimension vs Indexing Time (sec) (varying number of vectors) [ef_construction=150, M=16]

Indexing time increases with vector dimension across all three libraries: FAISS-HNSW, HNSWlib, and NMSLIB-HNSW. For a fixed number of vectors (10,000), ef_construction (150), and M (16), NMSLIB-HNSW consistently demonstrates the lowest indexing times, followed by FAISS-HNSW, with HNSWlib requiring the most time. For example, at 500 dimension, indexing times were approximately 0.3s (NMSLIB-HNSW), 0.37s (FAISS-HNSW), and 3.23s (HNSWlib). This trend remains consistent at larger vector counts and higher dimensions, with the gap between libraries widening as the dimensionality increases. The results suggest that NMSLIB-HNSW scales most efficiently with dimension, while HNSWlib incurs substantial computational overhead.

5.1.2 Number of Vectors vs. Indexing Time

Table 2: Number of Vectors vs Indexing Time (sec) (varying vector dimension) [ef_construction=150, M=16]

Dimension	Num of Vectors	Indexing Time (sec)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
100	10,000	0.1436	0.8415	0.0971
	50,000	0.9951	6.1740	0.6935
	100,000	2.2739	14.5105	1.4570
	500,000	19.4652	104.8404	14.4362
200	10,000	0.1962	1.3371	0.1628
	50,000	1.4490	10.6751	1.0252
	100,000	4.4077	26.4184	2.5743
	500,000	51.7080	185.7538	24.7186
300	10,000	0.2853	1.9584	0.2062
	50,000	2.3776	16.3359	1.4314
	100,000	7.7596	40.5838	3.9808
	500,000	77.3202	272.6696	33.6683
500	10,000	0.3687	3.2329	0.3019
	50,000	4.8166	27.2456	2.3567
	100,000	14.0627	64.6539	6.4599
	500,000	124.7015	420.5514	51.1286

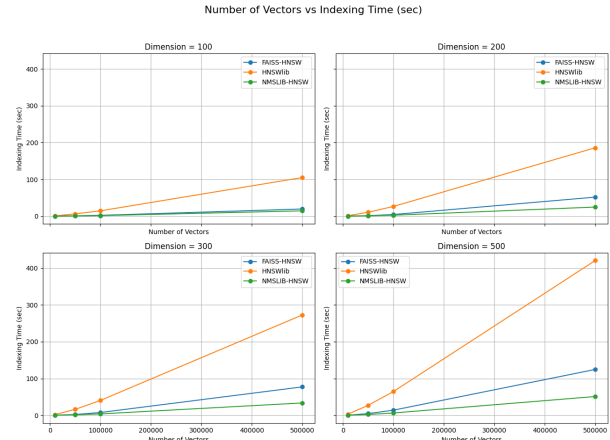


Figure 2: Number of Vectors vs Indexing Time (sec) (varying vector dimension) [ef_construction=150, M=16]

As the number of vectors increases, the indexing time increases nearly linearly for all three libraries. With dimension fixed at 100 and ef_construction and M held constant, NMSLIB-HNSW remains the fastest, while HNSWlib shows the steepest growth rate, especially at higher vector counts. For example, at 500,000 vectors and 500 dimension, indexing times reach approximately 51s (NMSLIB-HNSW), 125s (FAISS-HNSW), and 421s (HNSWlib). These differences highlight NMSLIB-HNSW's robust scalability and HNSWlib's higher sensitivity to dataset size.

5.1.3 ef_construction vs. Indexing Time

Table 3: ef_construction vs Indexing Time (sec) (varying value of M) [500K vectors, 500 dim]

M	ef_construction	Indexing Time (sec)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
8	50	20.3852	79.9888	9.5200
	100	35.7662	121.3579	15.6403
	150	49.5637	166.8701	21.5126
	300	91.7545	292.2005	39.6914
12	50	34.2864	139.0418	16.1317
	100	61.5815	212.8243	26.0606
	150	85.7136	279.8400	35.2520
	300	154.3961	460.5465	63.1347
16	50	48.4400	214.9253	23.2791
	100	89.1326	330.4500	38.0912
	150	124.7015	420.5514	51.1286
	300	221.1663	661.5787	88.6437
32	50	167.9966	552.1744	51.9668
	100	192.0881	1030.3402	96.5237
	150	280.8414	1325.9917	129.6906
	300	518.0934	1897.5970	211.2285

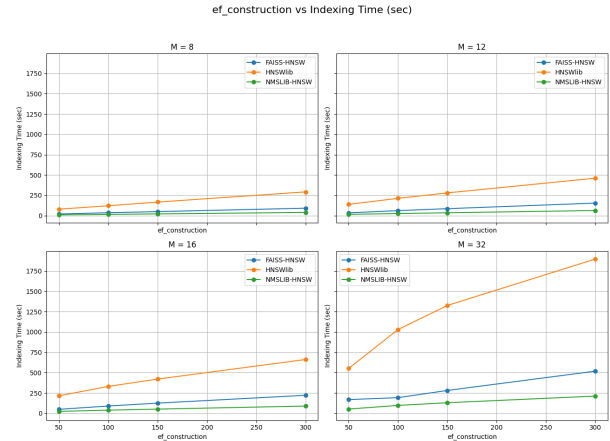


Figure 3: ef_construction vs Indexing Time (sec) (varying value of M) [500K vectors, 500 dim]

Indexing time grows sharply with increasing ef_construction for all libraries. This parameter, which controls the breadth of the graph construction search, significantly impacts computation: higher ef_construction values enable better graph connectivity but at the cost of slower indexing. The increase is most pronounced for HNSWlib: for example, at M=16, indexing time increases from ~215s to ~662s. FAISS-HNSW and NMSLIB-HNSW exhibit similar but less dramatic trends, e.g., for M=16, indexing time rises from ~48s to ~221s for FAISS and ~23s to ~89s for NMSLIB. This suggests that users seeking to optimize for indexing speed should carefully balance ef_construction based on their recall needs, particularly when using HNSWlib.

5.1.4 M vs. Indexing Time

Table 4: M vs Indexing Time (sec) (varying value of ef_construction) [500K vectors, 500 dim]

ef_construction	M	Indexing Time (sec)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
50	8	20.3852	79.9888	9.5200
	12	34.2864	139.0418	16.1317
	16	48.4400	214.9253	23.2791
	32	167.9966	552.1744	51.9668
100	8	35.7662	121.3579	15.6403
	12	61.5815	212.8243	26.0606
	16	89.1326	330.4500	38.0912
	32	192.0881	1030.3402	96.5237
150	8	49.5637	166.8701	21.5126
	12	85.7136	279.8400	35.2520
	16	124.7015	420.5514	51.1286
	32	280.8414	1325.9917	129.6906
300	8	91.7545	292.2005	39.6914
	12	154.3961	460.5465	63.1347
	16	221.1663	661.5787	88.6437
	32	518.0934	1897.5970	211.2285

Increasing the M parameter, which controls the maximum number of connections per node in the HNSW graph, leads to a substantial rise in indexing time. At the highest tested settings (500,000 vectors, 500 dimensions, ef_construction = 300), indexing time for HNSWlib increases from ~292s (M=8) to ~1,897s (M=32), while FAISS-HNSW goes from ~92s to 518s, and NMSLIB-HNSW from ~40s to 211s. This steep rise in computational demand reflects the increased graph complexity and is again most pronounced for HNSWlib.

5.1.5 Key observations

- NMSLIB-HNSW is consistently the fastest for indexing, with FAISS-HNSW intermediate, and HNSWlib the slowest.
- The impact of ef_construction and M is especially strong, suggesting users must carefully tune these for large scale data.
- Differences between libraries become most apparent as either dataset size, dimension, or parameter values increase.

5.2 Memory Used

5.2.1 Vector Dimension vs. Memory Used

Table 5: Vector Dimension vs Memory Used (MB) (varying number of vectors) [ef_construction=150, M=16]

Num of Vectors	Dimension	Indexing Time (sec)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
10,000	100	0.00	0.00	0.00
	200	0.00	0.00	0.56
	300	0.00	0.00	1.13
	500	0.00	0.00	0.00
50,000	100	0.00	0.00	1.13
	200	37.69	45.00	0.00
	300	56.81	64.69	0.56
	500	95.06	102.38	1.13
100,000	100	37.69	51.75	52.31
	200	75.38	90.56	91.69
	300	114.19	128.25	128.81
	500	190.69	204.75	204.75
500,000	100	253.69	257.63	261.56
	200	444.38	449.44	452.25
	300	635.06	639.00	642.94
	500	1016.44	1022.06	1023.75

For all libraries FAISS-HNSW, HNSWlib, and NMSLIB-HNSW, memory usage increases with vector dimension, especially for larger datasets. When the number of vectors is fixed at 10,000, the absolute memory consumption

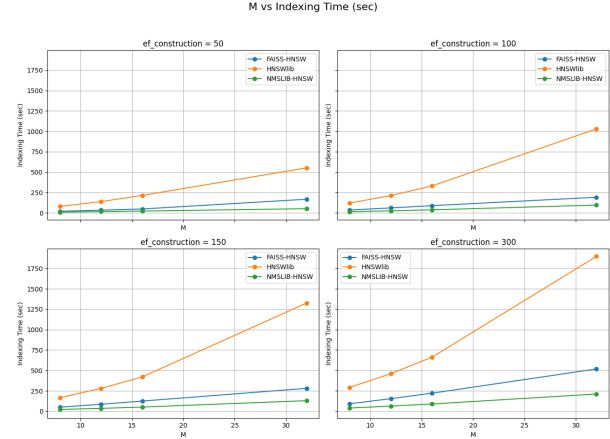


Figure 4: M vs Indexing Time (sec) (varying value of ef_construction) [500K vectors, 500 dim]

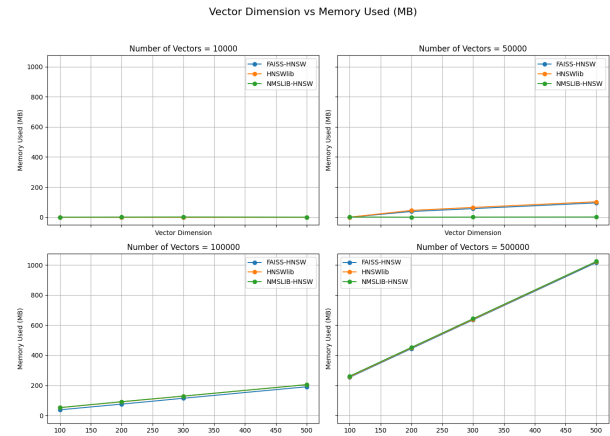


Figure 5: Vector Dimension vs Memory Used (MB) (varying number of vectors) [ef_construction=150, M=16]

is negligible, but as both dimension and dataset size grow, the memory consumption becomes more evident. For example, with 500,000 vectors and 500 dimensions, all three libraries used approximately 1,016–1,024 MB. Across all experiments, the memory usage difference between the libraries is marginal, typically within 1–2% of each other. This indicates that, in terms of memory footprint for storing vectors and index structures, the three implementations are quite similar and scale predictably with dimension.

5.2.2 Number of Vectors vs. Memory Used

Table 6: Number of Vectors vs Memory Used (MB) (varying vector dimension) [ef_construction=150, M=16]

Dimension	Num of Vectors	Memory Used (MB)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
100	10,000	0.00	0.00	0.00
	50,000	0.00	0.00	1.13
	100,000	37.69	51.75	52.31
	500,000	253.69	257.63	261.56
200	10,000	0.00	0.00	0.56
	50,000	37.69	45.00	0.00
	100,000	75.38	90.56	91.69
	500,000	444.38	449.44	452.25
300	10,000	0.00	0.00	1.13
	50,000	56.81	64.69	0.56
	100,000	114.19	128.25	128.81
	500,000	635.06	639.00	642.94
500	10,000	0.00	0.00	0.00
	50,000	95.06	102.38	1.13
	100,000	190.69	204.75	204.75
	500,000	1016.44	1022.06	1023.75

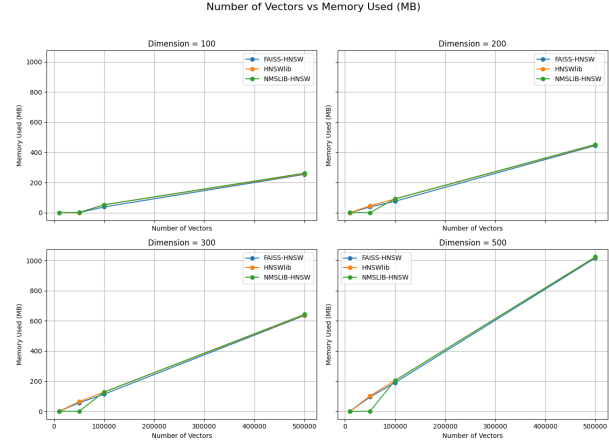


Figure 6: Number of Vectors vs Memory Used (MB) (varying vector dimension) [ef_construction=150, M=16]

Memory usage scales linearly with the number of vectors across all libraries. For a fixed dimension (100), memory used is minimal at low vector counts but increases steadily as the dataset size grows. For 500,000 vectors and 500 dimensions, the memory used by FAISS-HNSW, HNSWlib, and NMSLIB-HNSW is closely matched, each reporting about 1,016–1,024 MB. These results suggest that the index storage overhead is primarily driven by the number of vectors, with little distinction among the libraries.

5.2.3 ef_construction vs. Memory Used

Table 7: ef_construction vs Memory Used (MB) (varying value of M) [500K vectors, 500 dim]

M	ef_construction	Memory Used (MB)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
8	50	986.06	994.50	1312.31
	100	986.06	991.13	993.38
	150	986.06	990.00	992.81
	300	986.06	991.13	993.94
12	50	1001.25	1006.31	1009.13
	100	1001.25	1005.19	1009.69
	150	1001.25	1005.19	1009.69
	300	1001.25	1005.75	1008.56
16	50	1016.44	1020.38	1170.00
	100	1016.44	1020.94	1024.88
	150	1016.44	1022.06	1023.75
	300	1016.44	1020.38	1024.31
32	50	1077.19	1081.69	1085.63
	100	1077.19	1081.69	1084.50
	150	1077.19	1081.69	1085.63
	300	1077.19	1081.69	1085.06

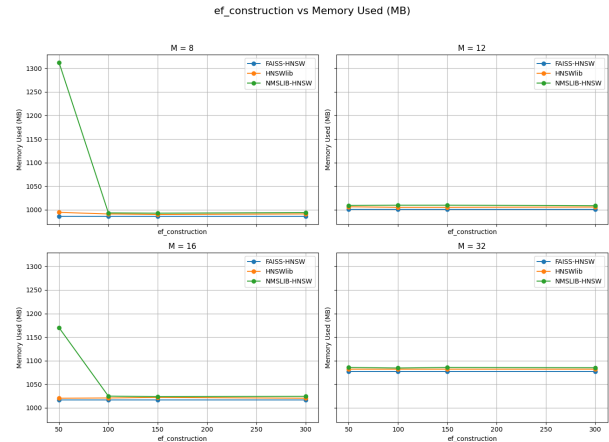


Figure 7: ef_construction vs Memory Used (MB) (varying value of M) [500K vectors, 500 dim]

Varying ef_construction does not significantly impact memory usage for any of the libraries. For a large dataset (500,000 vectors, 500 dimensions), increasing ef_construction from 50 to 300 results in memory usage that stays effectively constant, with all libraries fluctuating within a narrow band (e.g., ~1,077–1,086 MB). This demonstrates that ef_construction primarily influences computational cost during indexing and has negligible effect on the memory footprint of the constructed index.

5.2.4 M vs. Memory Used

Table 8: M vs Memory Used (MB) (varying value of ef_construction) [500K vectors, 500 dim]

ef_construction	M	Memory Used (MB)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
50	8	986.06	994.50	1312.31
	12	1001.25	1006.31	1009.13
	16	1016.44	1020.38	1170.00
	32	1077.19	1081.69	1085.63
100	8	986.06	991.13	993.38
	12	1001.25	1005.19	1009.69
	16	1016.44	1020.94	1024.88
	32	1077.19	1081.69	1084.50
150	8	986.06	990.00	992.81
	12	1001.25	1005.19	1009.69
	16	1016.44	1022.06	1023.75
	32	1077.19	1081.69	1085.63
300	8	986.06	991.13	993.94
	12	1001.25	1005.75	1008.56
	16	1016.44	1020.38	1024.31
	32	1077.19	1081.69	1085.06

The memory footprint increases moderately as the M parameter (maximum connections per node) is raised, since a denser graph requires more storage for additional links. For 500,000 vectors and 500 dimensions, memory usage for FAISS-HNSW, HNSWlib, and NMSLIB-HNSW grows from approximately 986–994 MB (M=8) to 1,077–1,086 MB (M=32), representing a ~9% increase across the tested range. Again, differences between libraries remain negligible, indicating similar underlying data structures and efficiency for this aspect of index storage.

5.2.5 Key observations

- Memory usage is largely determined by the number of vectors and dimension and increases in a predictable, nearly linear fashion.
- M has a moderate impact on memory use; ef_construction has almost none.
- All three libraries are nearly identical in memory usage, differing by at most a few percent even at extreme parameter settings.

5.3 Querying Time

5.3.1 Vector Dimension vs Querying Time

Table 9: Vector Dimension vs Querying Time (sec) (varying number of vectors) [ef_construction=150, M=16]

Num of Vectors	Dimension	Querying Time (sec)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
10,000	100	0.0075	0.0081	0.0247
	200	0.0083	0.0116	0.0396
	300	0.0113	0.0159	0.0579
	500	0.0099	0.0264	0.0865
50,000	100	0.0137	0.0498	0.1910
	200	0.0254	0.0957	0.3309
	300	0.0510	0.1404	0.4961
	500	0.0955	0.2204	0.8071
100,000	100	0.0250	0.1363	0.5140
	200	0.0937	0.2419	0.8876
	300	0.1490	0.3380	1.2210
	500	0.2978	0.5039	1.9795
500,000	100	0.3462	0.8454	4.1129
	200	0.9084	1.4139	6.4845
	300	1.2532	2.1994	8.8558
	500	1.9703	3.0583	12.5704

Querying time, measured as the total time taken to query 10% of all vectors, increases with vector dimension for all libraries. At smaller dimensions (e.g., 100), querying times are extremely low, typically under 0.03 seconds per query

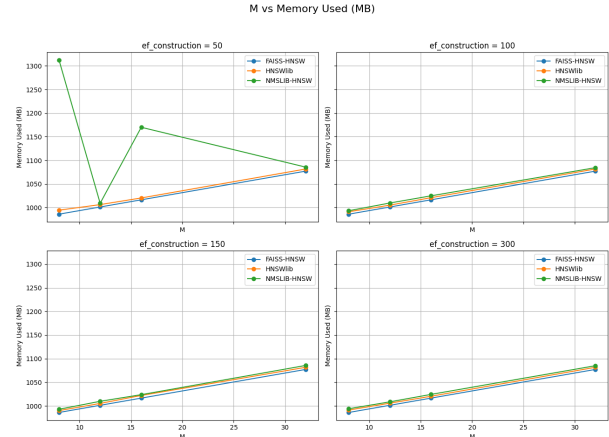


Figure 8: M vs Memory Used (MB) (varying value of ef_construction) [500K vectors, 500 dim]

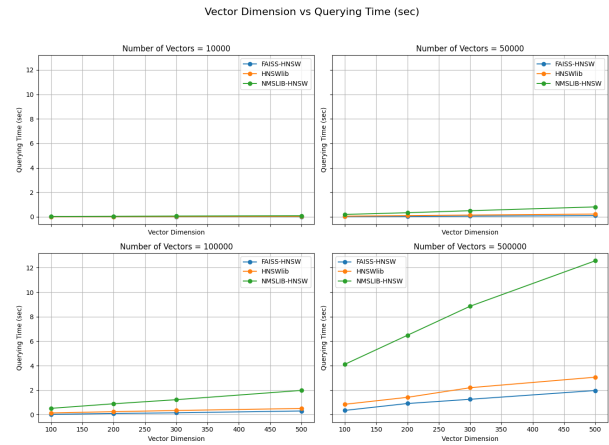


Figure 9: Vector Dimension vs Querying Time (sec) (varying number of vectors) [ef_construction=150, M=16]

for all libraries. As dimension rises to 500, querying time rises as well: for 500,000 vectors and 500 dimensions, FAISS-HNSW records about 1.97s, HNSWlib about 3.06s, and NMSLIB-HNSW about 12.57s. This trend holds across all tested configurations, with FAISS-HNSW typically positioned between the other two libraries.

5.3.2 Number of Vectors vs. Querying Time

Table 10: Number of Vectors vs Querying Time (sec) (varying vector dimensions) [ef_construction=150, M=16]

Dimension	Num of Vectors	Querying Time (sec)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
100	10,000	0.0075	0.0081	0.0247
	50,000	0.0137	0.0498	0.1910
	100,000	0.0250	0.1363	0.5140
	500,000	0.3462	0.8454	4.1129
200	10,000	0.0083	0.0116	0.0396
	50,000	0.0254	0.0957	0.3309
	100,000	0.0937	0.2419	0.8876
	500,000	0.9084	1.4139	6.4845
300	10,000	0.0113	0.0159	0.0579
	50,000	0.0510	0.1404	0.4961
	100,000	0.1490	0.3380	1.2210
	500,000	1.2532	2.1994	8.8558
500	10,000	0.0099	0.0264	0.0865
	50,000	0.0955	0.2204	0.8071
	100,000	0.2978	0.5039	1.9795
	500,000	1.9703	3.0583	12.5704

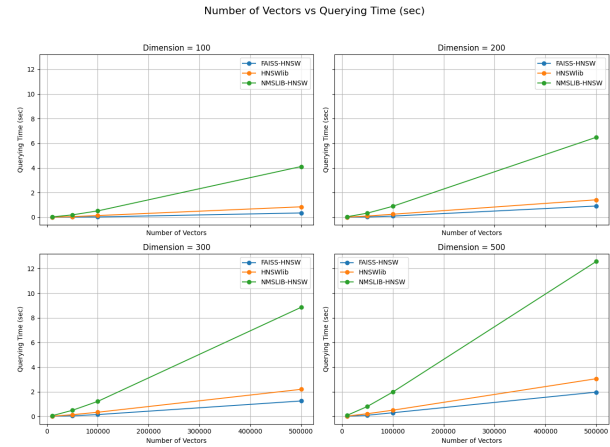


Figure 10: Number of Vectors vs Querying Time (sec) (varying vector dimensions) [ef_construction=150, M=16]

Querying time, measured as the total time taken to query 10% of all vectors, increases with the number of vectors in the dataset for all libraries. This trend is expected, as the total number of queries also increases proportionally with dataset size. It is important to note that querying time here reflects both the increase in dataset size and the increase in the number of queries performed. Comparisons of absolute querying time are meaningful only within the same dataset size and parameter setting. However, the rate of increase in querying time is more pronounced for NMSLIB-HNSW compared to FAISS-HNSW and HNSWlib as dataset size grows, indicating that NMSLIB-HNSW's query efficiency may degrade more rapidly at larger scales. Practitioners should be aware of this behavior, especially when considering large scale or high throughput retrieval scenarios.

5.3.3 ef_construction vs. Querying Time

Table 11: ef_construction vs Querying Time (sec) (varying value of M) [500K vectors, 500 dim]

M	ef_construction	Querying Time (sec)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
8	50	1.0001	1.4255	5.9654
	100	0.9925	1.4729	6.0544
	150	1.0131	1.5912	5.8744
	300	1.0331	1.5095	6.2014
12	50	1.4095	2.2734	9.0579
	100	1.4398	2.3180	9.4175
	150	1.4519	2.2661	8.9372
	300	1.5011	2.2621	9.2759
16	50	1.8464	2.9714	12.7617
	100	1.9499	2.9963	12.4333
	150	1.9703	3.0583	12.5704
	300	2.0155	3.1637	12.5256
32	50	3.6984	5.6201	23.2892
	100	3.9026	6.0111	24.9228
	150	4.0397	6.3043	25.1056
	300	4.1696	6.4708	26.3061

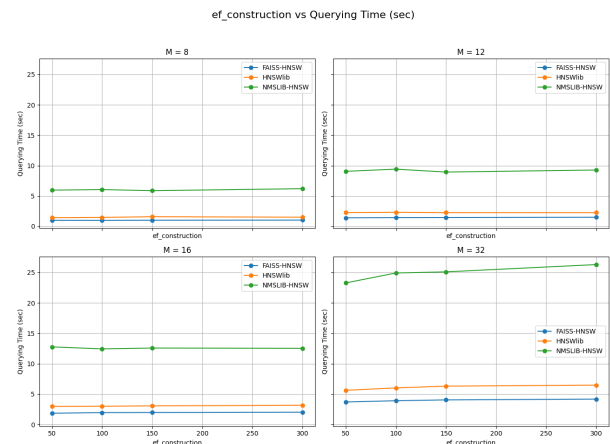


Figure 11: ef_construction vs Querying Time (sec) (varying value of M) [500K vectors, 500 dim]

Increasing the ef_construction parameter, which governs the search effort during indexing and querying, leads to a gradual rise in querying time for all libraries. For 500,000 vectors, 500 dimensions and M=32, querying time for FAISS-HNSW rises from ~3.7s (ef=50) to ~4.2s (ef=300), for HNSWlib from ~5.6s to ~6.5s, and for NMSLIB-HNSW from ~23.3s to ~26.3s. The effect is more pronounced for NMSLIB-HNSW, whose querying time grows faster than the other

libraries as ef_construction increases. This suggests a stronger trade-off between recall and speed for NMSLIB-HNSW when using higher ef_construction values.

5.3.4 M vs. Querying Time

Table 12: M vs Querying Time (sec) (varying value of M) [500K vectors, 500 dim]

ef_construction	M	Querying Time (sec)		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
50	8	1.0001	1.4255	5.9654
	12	1.4095	2.2734	9.0579
	16	1.8464	2.9714	12.7617
	32	3.6984	5.6201	23.2892
100	8	0.9925	1.4729	6.0544
	12	1.4398	2.3180	9.4175
	16	1.9499	2.9963	12.4333
	32	3.9026	6.0111	24.9228
150	8	1.0131	1.5912	5.8744
	12	1.4519	2.2661	8.9372
	16	1.9703	3.0583	12.5704
	32	4.0397	6.3043	25.1056
300	8	1.0331	1.5095	6.2014
	12	1.5011	2.2621	9.2759
	16	2.0155	3.1637	12.5256
	32	4.1696	6.4708	26.3061

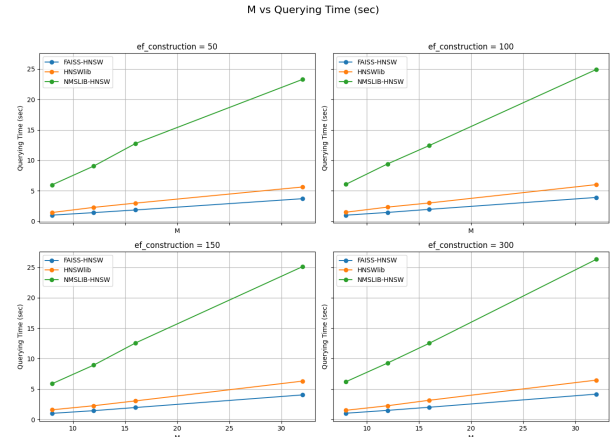


Figure 12: M vs Querying Time (sec) (varying value of M) [500K vectors, 500 dim]

Raising M (maximum number of connections per node) also increases querying time for all libraries. With 500,000 vectors, 500 dimensions, and ef_construction = 300, querying time for FAISS-HNSW rises from ~1.0s (M=8) to ~4.2s (M=32), for HNSWlib from ~1.5s to ~6.5s, and for NMSLIB-HNSW from ~6.0s to ~26.3s. Again, the increase is steeper for NMSLIB-HNSW at higher M values.

5.3.5 Key observations

- At moderate parameter settings, FAISS-HNSW and HNSWlib offer the lowest querying times, with NMSLIB-HNSW sometimes lagging as parameters grow.
- As parameter values and data scale increase, the gap in querying times becomes more pronounced, with NMSLIB-HNSW becoming significantly slower for high ef_construction or M.
- This highlights a trade-off in NMSLIB-HNSW: while it is very fast to index, its querying time can become less competitive at high recall/high complexity configurations.

5.4 Recall@10

5.4.1 Vector Dimension vs Recall@10

Table 13: Vector Dimension vs Recall@10 (varying number of vectors) [ef_construction=150, M=16]

Num of Vectors	Dimension	Recall@10		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
10,000	100	0.9090	0.7862	0.9607
	200	0.8103	0.6829	0.9162
	300	0.7635	0.6192	0.8912
	500	0.7149	0.5772	0.8608
50,000	100	0.6788	0.5371	0.8297
	200	0.4834	0.3609	0.6850
	300	0.4103	0.2961	0.6057
	500	0.3476	0.2421	0.5423
100,000	100	0.5407	0.4083	0.7317
	200	0.3489	0.2402	0.5431
	300	0.2816	0.1883	0.4639
	500	0.2320	0.1507	0.3957
500,000	100	0.2720	0.1751	0.4522
	200	0.1397	0.0785	0.2578
	300	0.1084	0.0579	0.2031
	500	0.0851	0.0435	0.1625

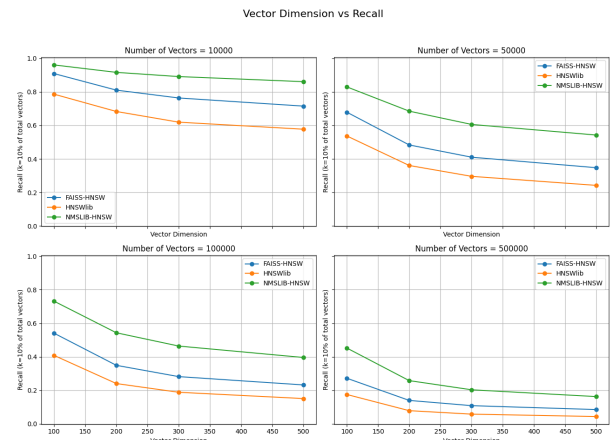


Figure 13: Vector Dimension vs Recall@10 (varying number of vectors) [ef_construction=150, M=16]

Recall@10 decreases as vector dimension increases for all three libraries. At lower dimensions (e.g., 100), NMSLIB-HNSW achieves the highest recall@10 (up to 0.96), followed by FAISS-HNSW (up to 0.91) and HNSWlib (up to 0.79). However, as the dimension increases to 500, recall@10 drops significantly, reaching 0.86 (NMSLIB-HNSW), 0.71 (FAISS-HNSW), and 0.58 (HNSWlib) for a small dataset (10,000 vectors, ef_construction=150, M=16). This trend is even more pronounced for larger datasets. The decrease in recall@10 with rising dimension highlights the inherent difficulty of approximate nearest neighbor search in higher dimensional spaces.

5.4.2 Number of Vectors vs. Recall@10

Table 14: Number of Vectors vs Recall@10 (varying vector dimension) [ef_construction=150, M=16]

Dimension	Num of Vectors	Recall@10		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
100	10,000	0.9090	0.7862	0.9607
	50,000	0.6788	0.5371	0.8297
	100,000	0.5407	0.4083	0.7317
	500,000	0.2720	0.1751	0.4522
200	10,000	0.8103	0.6829	0.9162
	50,000	0.4834	0.3609	0.6850
	100,000	0.3489	0.2402	0.5431
	500,000	0.1397	0.0785	0.2578
300	10,000	0.7635	0.6192	0.8912
	50,000	0.4103	0.2961	0.6057
	100,000	0.2816	0.1883	0.4639
	500,000	0.1084	0.0579	0.2031
500	10,000	0.7149	0.5772	0.8608
	50,000	0.3476	0.2421	0.5423
	100,000	0.2320	0.1507	0.3957
	500,000	0.0851	0.0435	0.1625

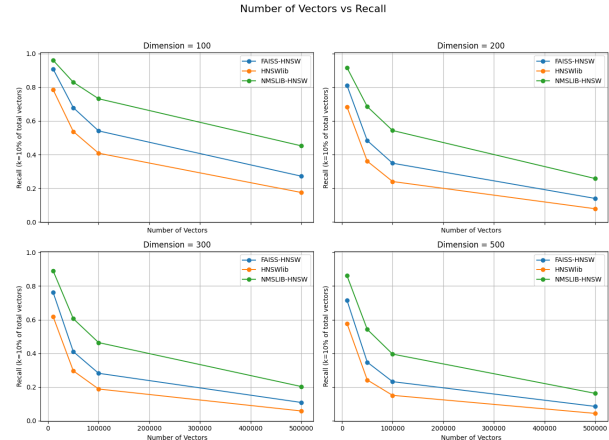


Figure 14: Number of Vectors vs Recall@10 (varying vector dimension) [ef_construction=150, M=16]

Recall@10 consistently declines as the number of vectors in the dataset increases, with all other parameters held constant. For example, at 500 dimension, ef_construction=150, and M=16, recall@10 drops from 0.86 (NMSLIB-HNSW), 0.71 (FAISS-HNSW), and 0.58 (HNSWlib) for 10,000 vectors to 0.16 (NMSLIB-HNSW), 0.09 (FAISS-HNSW), and 0.04 (HNSWlib) for 500,000 vectors. This trend reflects the increased difficulty of finding true nearest neighbors as the dataset grows. Nevertheless, NMSLIB-HNSW consistently outperforms the other two libraries, achieving the highest recall@10 at all dataset sizes.

5.4.3 ef_construction vs. Recall@10

Table 15: ef_construction vs Recall@10 (varying value of M) [500K vectors, 500 dim]

M	ef_construction	Recall@10		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
8	50	0.0340	0.0124	0.0616
	100	0.0335	0.0151	0.0725
	150	0.0335	0.0174	0.0794
	300	0.0340	0.0200	0.0913
12	50	0.0531	0.0200	0.0932
	100	0.0563	0.0259	0.1110
	150	0.0584	0.0293	0.1222
	300	0.0619	0.0366	0.1427
16	50	0.0710	0.0274	0.1226
	100	0.0800	0.0374	0.1479
	150	0.0851	0.0435	0.1625
	300	0.0938	0.0550	0.1926
32	50	0.1400	0.0548	0.2040
	100	0.1636	0.0871	0.2679
	150	0.1906	0.1062	0.3005
	300	0.2316	0.1392	0.3580

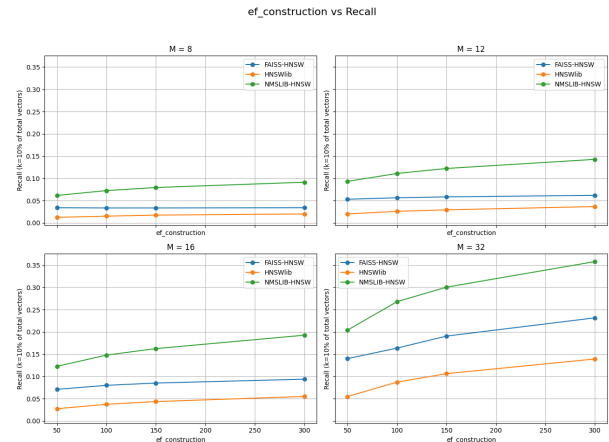


Figure 15: ef_construction vs Recall@10 (varying value of M) [500K vectors, 500 dim]

Recall@10 improves markedly as ef_construction increases for all libraries. At 500,000 vectors, 500 dimension and M=32, raising ef_construction from 50 to 300 boosts recall@10 from 0.20 to 0.36 (NMSLIB-HNSW), 0.14 to 0.23 (FAISS-HNSW), and 0.05 to 0.14 (HNSWlib). This demonstrates that a higher ef_construction enables more thorough graph traversal during search, thus yielding higher recall@10, especially important for large and complex datasets.

5.4.4 M vs. Recall@10

Table 16: M vs Recall@10 (varying value of M) [500K vectors, 500 dim]

ef_construction	M	Recall@10		
		FAISS-HNSW	HNSWlib	NMSLIB-HNSW
50	8	0.0340	0.0124	0.0616
	12	0.0531	0.0200	0.0932
	16	0.0710	0.0274	0.1226
	32	0.1400	0.0548	0.2040
100	8	0.0335	0.0151	0.0725
	12	0.0563	0.0259	0.1110
	16	0.0800	0.0374	0.1479
	32	0.1636	0.0871	0.2679
150	8	0.0335	0.0174	0.0794
	12	0.0584	0.0293	0.1222
	16	0.0851	0.0435	0.1625
	32	0.1906	0.1062	0.3005
300	8	0.0340	0.0200	0.0913
	12	0.0619	0.0366	0.1427
	16	0.0938	0.0550	0.1926
	32	0.2316	0.1392	0.3580

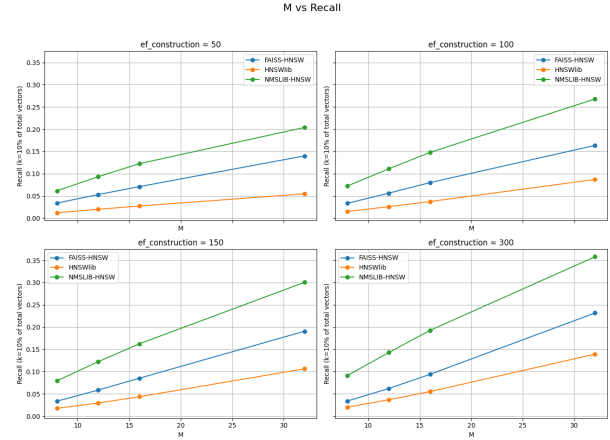


Figure 16: M vs Recall@10 (varying value of M) [500K vectors, 500 dim]

Increasing the parameter M (the maximum number of connections per node) results in higher recall@10 for all libraries. For the most demanding configuration (500,000 vectors, 500 dimensions), recall@10 increases from 0.09 to 0.36 for NMSLIB-HNSW, 0.03 to 0.23 for FAISS-HNSW, and 0.02 to 0.14 for HNSWlib as M increases from 8 to 32. This indicates that denser connectivity within the HNSW graph improves the ability to retrieve true nearest neighbors.

5.4.5 Key observations

- Recall@10 consistently decreases with higher vector dimensions and larger datasets, but can be improved by increasing ef_construction and M.
- Across all settings, NMSLIB-HNSW achieves the highest recall@10, with FAISS-HNSW and HNSWlib following.
- Careful tuning of ef_construction and M is crucial for maintaining high recall@10, especially at scale.

6 Discussion

Our comprehensive benchmarking of FAISS-HNSW, HNSWlib, and NMSLIB-HNSW reveals the nuanced trade-offs practitioners must consider when selecting and tuning HNSW-based libraries for approximate nearest neighbor (ANN) search. Although all three implementations are based on the same core algorithm, differences in engineering decisions, parameter defaults, and ecosystem integration lead to distinct performance profiles.

Indexing Time: NMSLIB-HNSW consistently provides the fastest index construction, making it well suited for use cases where rapid indexing is a priority. FAISS-HNSW offers intermediate indexing times, and HNSWlib is slower, particularly as dataset size, dimensionality, or key parameters (ef_construction, M) increase.

Memory Usage: All three libraries exhibit nearly identical memory footprints, scaling predictably and linearly with the number and dimension of the vectors. Memory usage is largely unaffected by ef_construction or M, so efficiency is not a distinguishing factor in this respect.

Querying Time: FAISS-HNSW and HNSWlib offer the lowest querying times across all tested configurations, whereas NMSLIB-HNSW exhibits higher query latency, particularly as ef_construction or M increase. This highlights a speed-accuracy trade-off: increasing parameters to boost recall also increases query times, most noticeably for NMSLIB-HNSW.

Recall@10: NMSLIB-HNSW achieves the highest recall@10 in all tested settings, with FAISS-HNSW close behind and HNSWlib trailing. Recall@10 decreases for all libraries as the vector dimension or dataset size increases. Both ef_construction and M can be tuned to improve recall@10, but at the cost of increased indexing and query times.

6.1 Practical Recommendations

- For fast indexing with high recall needs, especially when query latency is less critical, NMSLIB-HNSW is preferred.

- For balanced performance and integration with the FAISS ecosystem, FAISS-HNSW is the most versatile choice.
- For smaller scale tasks where simplicity and predictable behavior are valued, HNSWlib is a suitable option, but it is less optimal for large or high-dimensional datasets.
- Parameter tuning (increasing `ef_construction` and `M`) is the most effective way to improve recall, though this comes at the cost of slower indexing and query times, particularly for NMSLIB-HNSW.
- Memory usage is comparable across all libraries and is determined mainly by vector count and dimension.

6.2 Limitations

This study was conducted on synthetic datasets using in-memory indexes. Results may differ for real-world data, disk-based indexes, or with hardware-specific optimizations. Furthermore, the experiments focused exclusively on CPU-based libraries and did not evaluate GPU accelerated indexers, such as FAISS’s GPU implementation, which may offer significantly different performance characteristics. The analysis also focused on commonly used configurations; further fine-tuning or custom engineering could yield different results.

7 Future Directions

Future work could include benchmarking additional ANN algorithms, such as recent graph-based methods (e.g. DiskANN [5]) and hybrid quantization-based systems like ScaNN [6], as well as transformer-based similarity models. Another promising direction is to evaluate the performance of tree-based methods such as Annoy [7], which provides an alternative to graph-based and quantization-based approaches. Evaluating performance in large-scale real-world datasets (e.g. GloVe[9], SIFT1M[10], or OpenAI’s text embeddings[11]) would also improve the external validity of results.

Further comparisons should explore the behavior of both in-memory and disk-backed ANN systems under different latency and throughput requirements. Additionally, the integration of ANN search into vector databases such as Milvus [8] represents a promising direction for applied research in large-scale vector management.

Lastly, exploring GPU-accelerated implementations and performance across diverse hardware platforms would better reflect production deployment scenarios.

8 Conclusion

In this study, we presented a comprehensive benchmarking and comparative analysis of three widely used HNSW based approximate nearest neighbor (ANN) libraries: FAISS-HNSW, HNSWlib, and NMSLIB-HNSW. By systematically varying dataset sizes, vector dimensions and core HNSW parameters (`ef_construction` and `M`), we evaluated each library across four key metrics: indexing time, memory usage, querying time and recall.

Our experiments demonstrate clear performance trade-offs among the libraries. NMSLIB-HNSW stands out for its exceptionally fast index construction and consistently high recall, although it incurs higher query latencies, especially at more aggressive parameter settings. FAISS-HNSW delivers balanced performance across all metrics and is well suited for general purpose applications, particularly when integration with other FAISS features is desirable. HNSWlib offers stable and straightforward usage for small to moderate datasets but is less optimal for large-scale or high-dimensional workloads due to slower indexing and querying.

We show that parameter tuning, especially increasing `ef_construction` and `M`, can significantly improve recall but at the cost of increased computational overhead. All three libraries have similar memory footprints, with memory usage scaling predictably with dataset size and dimension.

Overall, our findings provide practical guidance for choosing and configuring HNSW libraries for diverse real-world workloads. Future work could extend this benchmarking to include additional libraries, disk-based indexes, or real-world datasets from specific domains.

References

- [1] Malkov, Y.A.; and Yashunin, D.A.
Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.
IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836, 2020.
DOI: <https://doi.org/10.1109/TPAMI.2018.2889473>
Code available at: <https://github.com/nmslib/hnswlib>
- [2] Johnson, J.; Douze, M.; and Jégou, H.
Billion-Scale Similarity Search with GPUs.
arXiv preprint arXiv:1702.08734, 2017.
DOI: <https://doi.org/10.48550/arXiv.1702.08734>
Code available at: <https://github.com/facebookresearch/faiss>
- [3] Boytsov, L.; and Naidan, B.
Engineering Efficient and Effective Non-Metric Space Library.
In *Similarity Search and Applications (SISAP)*, pp. 280–293, 2013.
DOI: https://doi.org/10.1007/978-3-642-41062-8_28
Code available at: <https://github.com/nmslib/nmslib>
- [4] Aumüller, M.; Bernhardsson, E.; and Faithfull, A.
ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms.
arXiv preprint arXiv:1807.05614, 2018.
DOI: <https://doi.org/10.48550/arXiv.1807.05614>
- [5] Subramanya, S. J.; Devvrit; Kadekodi, R.; Krishnaswamy, R.; and Simhadri, H. V.
DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node.
In *Proceedings of the 33rd Conference on Neural Information Processing Systems (NeurIPS '19)*, Article 1233,
pp. 13766–13776, 2019.
URL: <https://papers.nips.cc/paper/9527>
Code available at: <https://github.com/microsoft/DiskANN>
- [6] Guo, R.; Sun, P.; Lindgren, E.; Geng, Q.; Simcha, D.; Chern, F.; and Kumar, S.
Accelerating Large-Scale Inference with Anisotropic Vector Quantization.
arXiv preprint arXiv:1908.10396, 2020.
DOI: <https://doi.org/10.48550/arXiv.1908.10396>
- [7] Bernhardsson, E.
Annoy: Approximate Nearest Neighbors in C++/Python.
Code available at: <https://github.com/spotify/annoy>
- [8] Wang, J.; Yi, X.; Guo, R.; Jin, H.; et al.
Milvus: A Purpose-Built Vector Data Management System.
In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, pp. 2614–2627,
2021.
DOI: <https://doi.org/10.1145/3448016.3457550>
- [9] Pennington, J.; Socher, R.; and Manning, C. D.
GloVe: Global Vectors for Word Representation.
In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp.
1532–1543, 2014.
DOI: <https://doi.org/10.3115/v1/D14-1162>
Available at: <https://nlp.stanford.edu/projects/glove/>
- [10] Jégou, H.; Douze, M.; and Schmid, C.
Product Quantization for Nearest Neighbor Search.
In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1): 117–128, 2011.
Dataset: SIFT1M.
DOI: <https://doi.org/10.1109/TPAMI.2010.57>
- [11] OpenAI.
OpenAI Text Embeddings.
Available at: <https://platform.openai.com/docs/guides/embeddings>
- [12] Shah, Harshit.
HNSW Comparison. GitHub repository, 2025.
Available at: https://github.com/harshit7999/hnsw_comparison